

Hospital Management System

Features and Functionality

- **Patient Profile Management:**
 - The System allows the management of patient profiles, including their personal information, medical history and current treatment plan.
- **Doctor Profile Management:**
 - The System allows the management of doctor profiles, including their personal information, qualifications and specialties.
- **Appointment Scheduling:**
 - The System allows the scheduling and rescheduling of appointments with automated notifications to both patients and doctors.
- **Billing and Payment Management:**
 - The System allows the management of billing and payments with automated invoice generation and tracking of payments.

API Calls for the Hospital Management System:

For Patient:

POST /patient: Creates a new patient record

GET /patient: Retrieves a list of all patients

GET /patient/:id: Retrieves a specific patient's details by their ID

PUT /patient/:id: Updates an existing patient's details by their ID

DELETE /patient/:id: Deletes a specific patient's record by their ID

For Doctor:

POST /doctor: Creates a new doctor record

GET /doctor: Retrieves a list of all doctors

GET /doctor/:id: Retrieves a specific doctor's details by their ID

PUT /doctor/:id: Updates an existing doctor's details by their ID

DELETE /doctor/:id: Deletes a specific doctor's record by their ID

For Appointment:

POST /appointment: Creates a new appointment record

GET /appointment: Retrieves a list of all appointments

GET / appointment /:id: Retrieves a specific appointment's details by their ID

PUT / appointment /:id: Updates an existing appointment's details by their ID

DELETE / appointment /:id: Deletes a specific appointment's record by their ID

For Billing:

POST /bill: Creates a new bill record

GET / bill: Retrieves a list of all bills

GET / bill /:id: Retrieves a specific bill's details by their ID

PUT / bill /:id: Updates an existing bill's details by their ID

DELETE / bill /:id: Deletes a specific bill's record by their ID

Lombok

- Lombok is a library that helps to reduce boilerplate code in Java classes by automatically generating getter, setters, constructor and other common methods.

Loj4j2 Library

- Widely used logging library in the Java ecosystem
- Performant
- Flexibility
- Logger Interface
 - Error
 - Warn
 - Info
 - Debug

Optional

Container object may or may not contain non-null values.

Pagination

Pagination is a technique used to divide a large set of data into smaller, manageable chunks or pages. Instead of displaying all records at once, you show a

limited number of records per page. This is particularly useful for improving performance and user experience when dealing with large datasets.

Key Terms:

1. **Page:** A subset of the total records.
2. **Page Size:** The number of records displayed on each page.
3. **Current Page:** The page number currently being viewed.
4. **Total Pages:** The total number of pages available based on the dataset and page size.

Explanation of Response Fields

- **content:** An array of patient for the current page.
- **pageable:** Metadata about the pageable request.
- **last:** Indicates if this is the last page.
- **totalElements:** The total number of records in the dataset.
- **totalPages:** The total number of pages available.
- **size:** The number of records per page(page size).
- **Number:** The current page number(starting from 0).
- **first:** Indicates if this is the first page.
- **numberOfElements:** The number of records in the current page.
- **empty:** Indicates if the current page is empty.

Pagination in Spring Boot

Spring Boot provides built-in support for pagination via in Spring Data JPA repository. The core concept involves the Pageable interface and the Page class.

Key Components:

1. **Pageable:** Interface for pagination information (like page number and size).
2. **Page:** Interface for a page of data, including the content and pagination metadata.

Monitoring

Monitoring refers to the process of continuously observing and checking the performance, health and functionality of an application or system. The goal is ensure the system operates as expected to detect issues promptly and to gather data for analysis and improvement. Monitoring typically involves tracking metrics such as CPU usage, response times, error rates and application logs.

Monitoring with Respect to Spring Boot

In Spring Boot monitoring involves using various tools and techniques to track the performance and health your application. Spring Boot provides several built-in tools to facilitate monitoring including:

1. **Spring Boot Actuator:** A set built-in endpoints that insights into the application.
2. **Micrometer:** An application metrics façade to integrate with various monitoring system.
3. **External Monitoring Tools:** Tools like Prometheus, Grafana, ELK Stack(Elasticsearch, Logstash, Kibana) and others can be used to collect and visualize metrics and logs.

Webhooks:

Webhooks are automated messages sent from apps when something happens. They have a message (or payload) and are sent to a unique URL (Webhook URL) define by the user.

Usage: Commonly used to send real-time data to other applications. For example, GitHub sends webhooks when code is pushed and payment gateways send webhooks when a transaction is completed.

Example:

A webhook URL might look like: <https://yourapp.com/webhook>

When an event occurs, an HTTP POST request with event data is sent to this URL.

Order food → swiggy(unique URL) → redirected to payment gateway → successful payment

Use case:

Whenever you try to schedule an appointment and if the appointment is successfully scheduled, I want to send a automated message on the webhook URL.

- Webhook Producer → unique URL → <http://localhost:8081/webhook>